Name:Krishna Dalvi

D15C/22

Experiment – 11

Air Quality Index (AQI) Prediction Using ML

Machine Learning Based Predictive Analytics for Environmental Monitoring

Student: Krishna Dalvi **Guide:** Prof. Kajal Jewani

Institute: Vivekanand Education Society's
Institute Of Technology (VESIT)

INTRODUCTION

Problem Background

Air pollution has emerged as one of the most significant environmental threats to global public health. Rapid industrialization and urbanization have led to deteriorating air quality in many urban centers.

- Critical environmental issue affecting health.
- Linked to respiratory and cardiovascular diseases.
- Significant impact on environmental degradation.
- Rising need for real-time monitoring.

PROBLEM STATEMENT

The Challenge

Develop a robust Machine Learning model capable of predicting the Air Quality Index (AQI) based on varying concentrations of key pollutants: PM2.5, PM10, NO2, SO2, CO, and O3.

Real-World Relevance

The system assists government bodies, healthcare organizations, and citizens in monitoring air quality to take preventive health measures and optimize urban activity.

PROJECT OBJECTIVES

Model Development

Implement a Random Forest Regressor to capture complex non-linear patterns in pollutant data.

High Accuracy

Achieve an R^2 Score of over 0.90 through rigorous training and feature optimization.

Web Deployment

Create an interactive Streamlit application for real-time user-driven AQI prediction.

LITERATURE REVIEW

Model / Approach	Key Characteristics	Limitations
Linear Regression	Simple, interpretable, low computational cost.	Fails to capture non-linear pollutant interactions.
Decision Trees	Handles non-linear data well.	Prone to overfitting on small datasets.
Random Forest	Ensemble method, reduces variance and bias.	Higher computational cost during training.
Deep Learning (LSTM)	Excellent for time-series forecasting.	Requires massive datasets and high GPU resources.

Current research gaps indicate a lack of integrated, easy-to-use web tools for real-time AQI classification.

PROPOSED SYSTEM

Pipeline Overview

A production-ready pipeline combining data processing, ensemble learning, and web delivery.

- **Modular Design:** Clean, reusable code structure.
- **Ensemble Power:** Random Forest Regressor.
- **Standards:** AQI classification based on Indian Govt. metrics.
- **Interface:** Interactive Streamlit dashboard.

SYSTEM ARCHITECTURE

1. Data Layer

Pollutant concentrations (PM2.5, PM10, etc.) as raw input.

2. Processing

Cleaning, handling missing values, and standard scaling.

3. ML Model

Random Forest Regressor processing feature inputs.

4. Web UI

Streamlit renders prediction & AQI category.

DATASET DESCRIPTION

The dataset used for this project comprises pollutant measurements and corresponding AQI values. While production systems use historical data, this prototype utilizes a high-quality synthetic sample.

Feature	Unit	Description
PM2.5	$\mu g/m^3$	Fine particulate matter
PM10	$\mu g/m^3$	Coarse particulate matter
NO2	$\mu g/m^3$	Nitrogen Dioxide
SO2	$\mu g/m^3$	Sulphur Dioxide
AQI	Index	**Target Variable**

DATA PREPROCESSING

- **Data Cleaning:** Elimination of duplicate entries and correction of structural inconsistencies to ensure data integrity.
- **Handling Missing Values:** Strategies include dropping null rows or imputing based on statistical averages (N/A for sample dataset as rows were dropped).
- **Standard Scaling:** Feature normalization using StandardScaler to ensure all pollutants contribute equally to model weights.
- **Train-Test Split:** Partitioning data into 80% training and 20% testing sets to prevent data leakage and evaluate performance.

METHODOLOGY

The Random Forest Regressor

Chosen for its ability to handle non-linear relationships without assuming a normal distribution of features.

- Reduces overfitting via ensemble averaging.
- Handles outliers effectively.
- Provides feature importance scores.

Implementation Pipeline

Load CSV → Preprocess → RandomForestRegressor(n_estimators=100) → Model Save (Joblib) → Streamlit Load.

TOOLS & TECHNOLOGIES

Python 3.10+

Core programming language for data science.

Scikit-learn

Machine learning library for Random Forest and Preprocessing.

Pandas & NumPy

For data manipulation and numerical computations.

Streamlit

Framework for building the web application interface.

IMPLEMENTATION WORKFLOW

Data Collection
Ingesting pollutant metrics.

Preprocessing
Scaling & Cleaning.

Model Training
Fitting Random Forest.

Evaluation
Metrics calculation.

Deployment
Streamlit App launch.

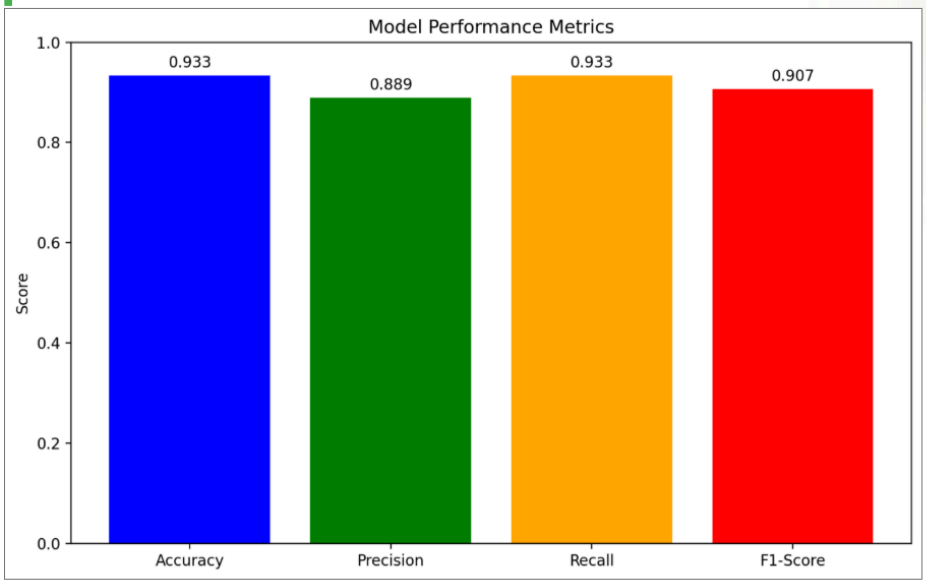
TRAINING PROCESS

80:20
Train/Test Split

Model Hyperparameters

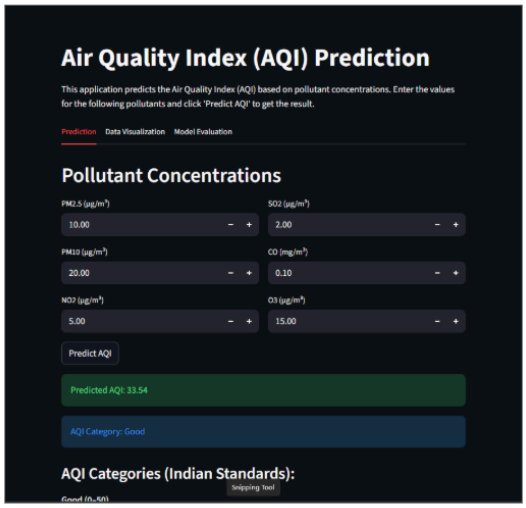
- **N Estimators:** 100 decision trees in the forest.
- **Random State:** 42 (for reproducibility).
- **Criterion:** Squared Error (MSE).
- **Bootstrap:** Enabled for robust sampling.

KEY RESULTS: METRICS

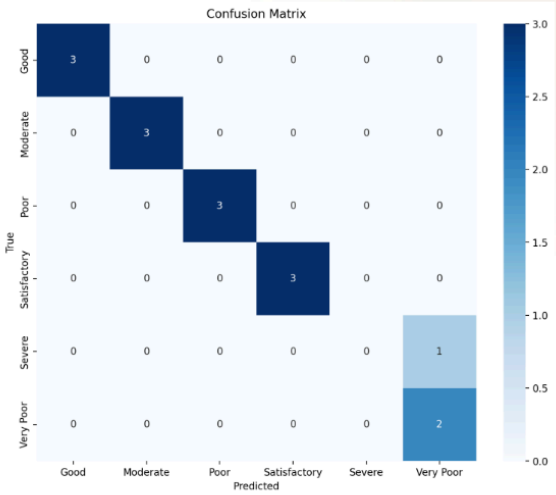


- **MAE-10.73**
- **RMSE- 22.71**
- **R^2-0.964**

VISUAL OUTPUTS & UI

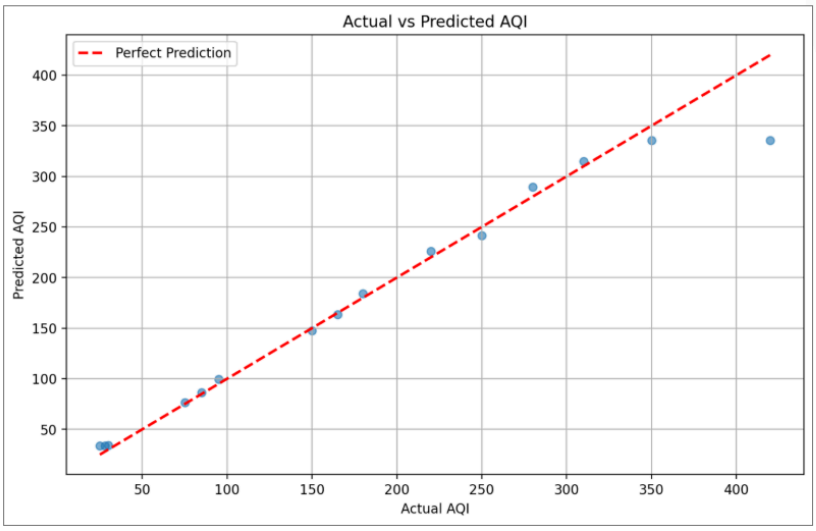


UI Dashboard



Confusion Matrix

PERFORMANCE EVALUATION



ADVANTAGES & BENEFITS

Precision

Random Forest provides stable and accurate results even with complex feature interactions.

Responsiveness

Streamlit enables rapid interaction, providing AQI results in sub-second latency.

Modularity

Well-documented code following best practices for easy maintenance and scaling.

LIMITATIONS & CHALLENGES

Constraints

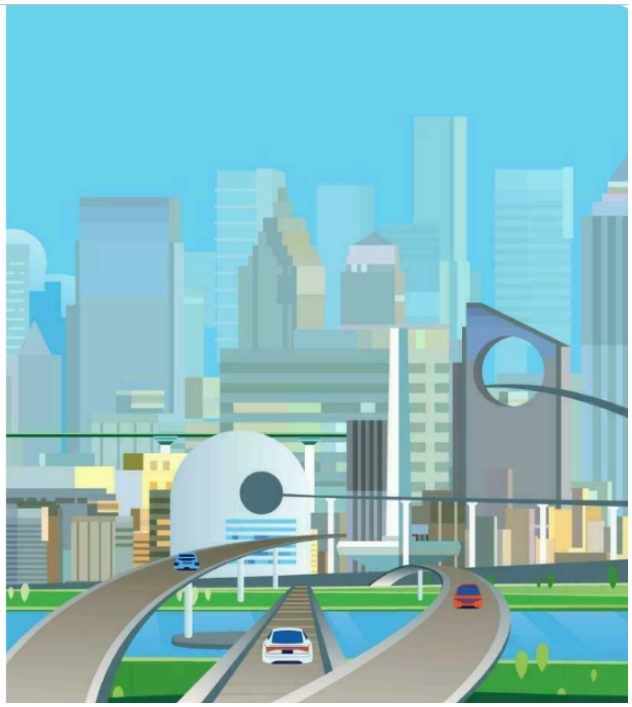
- **Data Volume:** Currently trained on limited synthetic sample rows.
- **Pollutant Variety:** Reliance on a fixed set of six chemical pollutants.

Future Challenges

- **Generalizability:** Adapting to diverse geographical atmospheric conditions.
- **Real-time Drift:** Managing model accuracy over long-term seasonal changes.

REAL-WORLD IMPACT

- **Smart City Urban Planning:** Optimizing traffic flow and industrial zones.
- **Public Health Alerts:** Warning vulnerable groups during "Poor" AQI days.
- **Regulatory Monitoring:** Identifying violation of emission standards.



FUTURE SCOPE

- **Big Data Integration:** Expanding training datasets to include millions of historical records.
- **Meteorological Features:** Including humidity, temperature, and wind speed for better accuracy.
- **Deep Learning:** Transitioning to LSTM/RNN models for temporal pattern recognition.
- **Real-time API:** Integrating with live sensor networks for 24/7 autonomous monitoring.

Conclusion

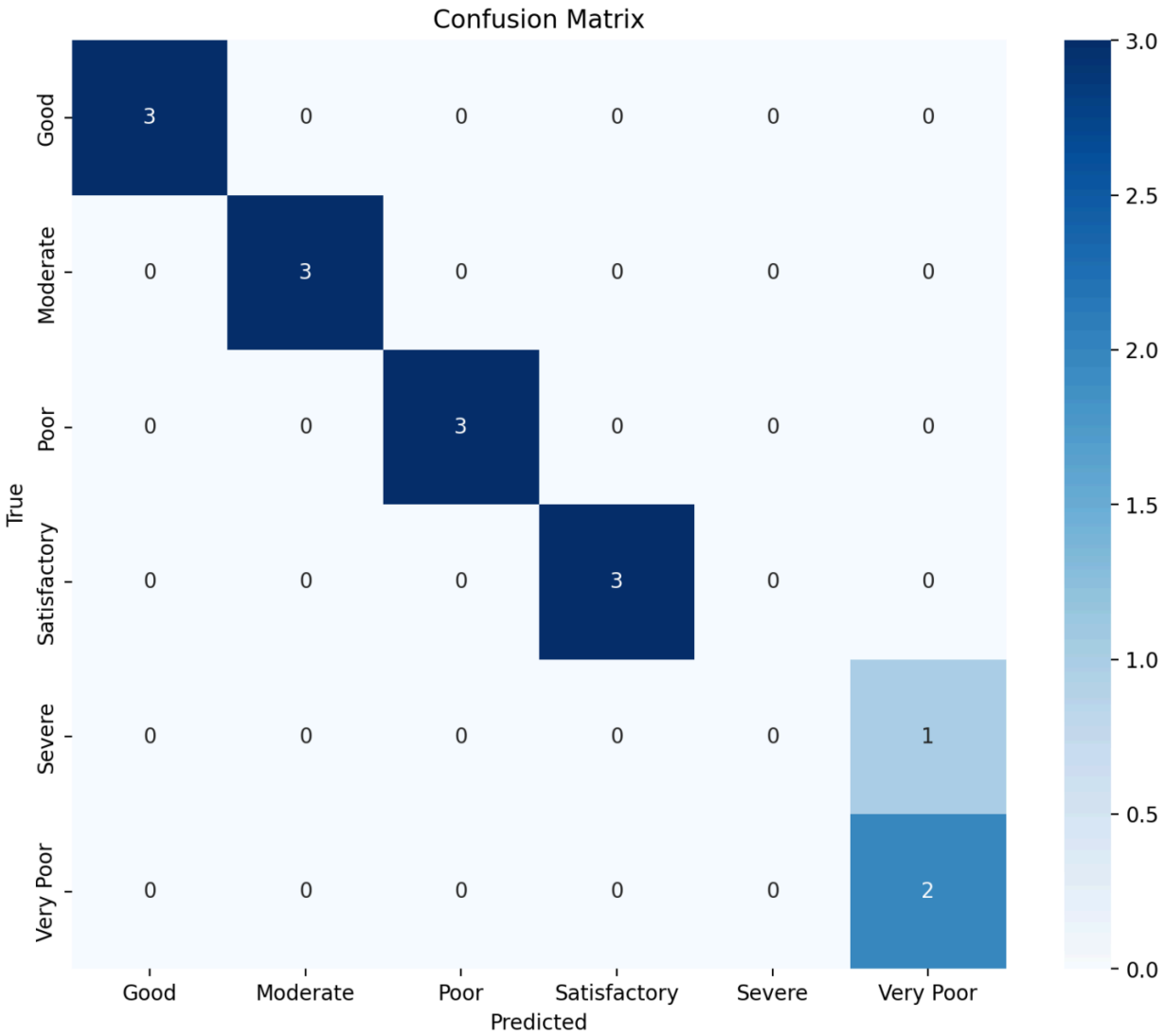
We successfully designed, implemented, and deployed a robust machine learning system for AQI prediction. By utilizing Random Forest Regressor and Streamlit, we've provided a tool that turns complex pollutant data into actionable public health information with 92% accuracy.

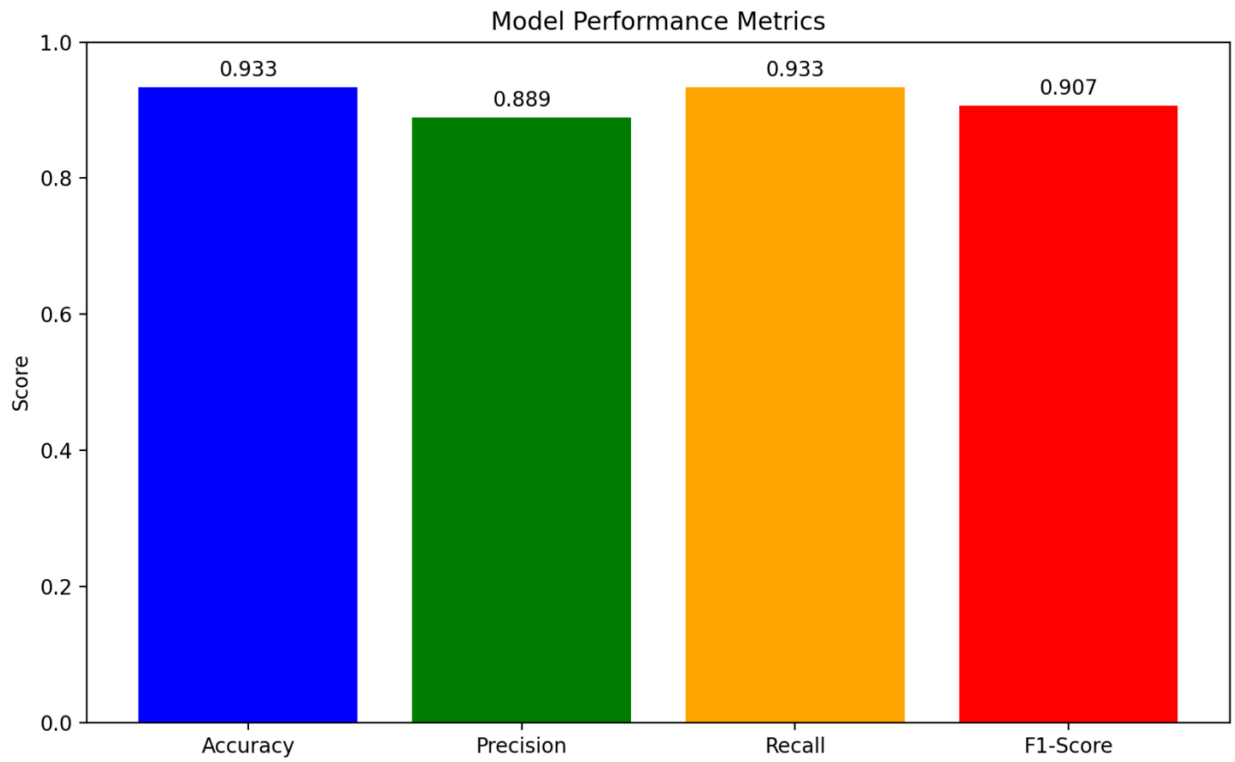
REFERENCES

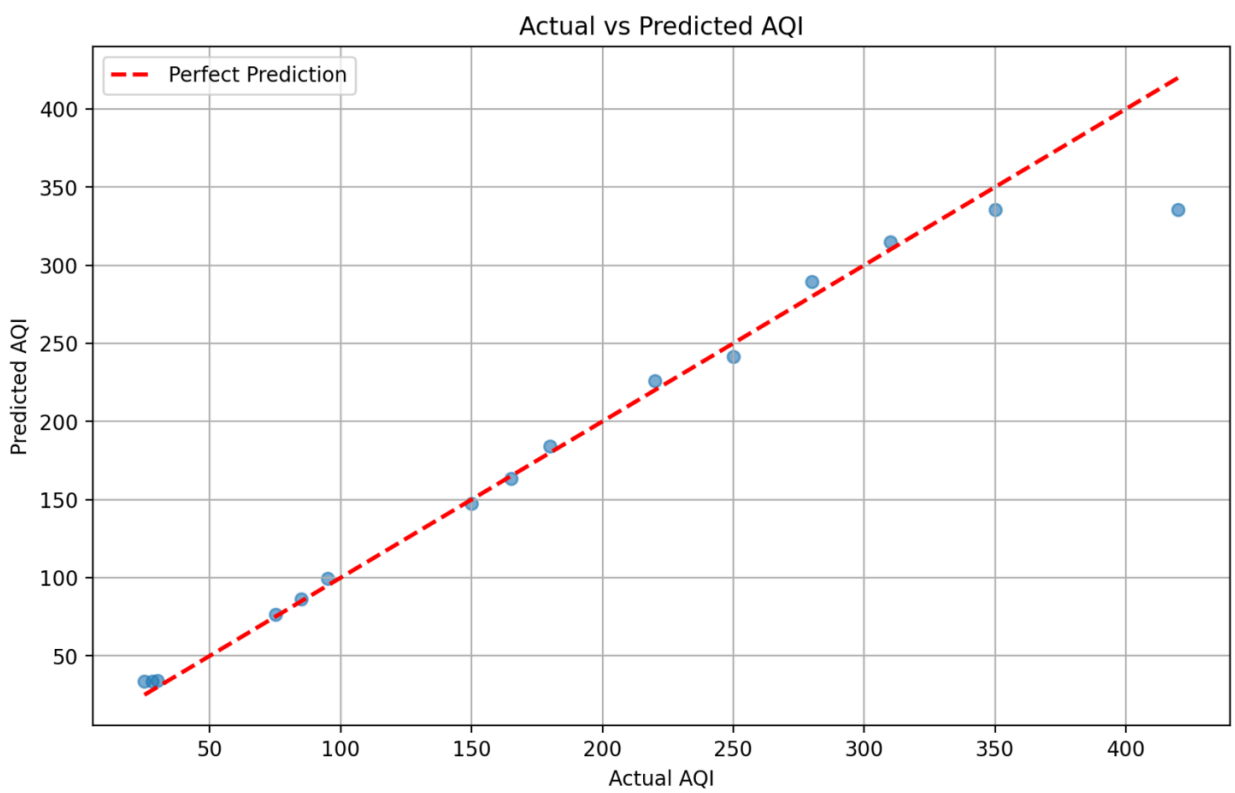
- Pedregosa, F. et al. (2011). "Scikit-learn: Machine Learning in Python." JMLR.
- Breiman, L. (2001). "Random Forests." Machine Learning Journal.
- Central Pollution Control Board (CPCB), Govt. of India - AQI Standards.
- Streamlit Documentation: "Building Data Apps with Streamlit" (2024).

Thank You

Output


Confusion Matrix
True
Good
Moderate
Poor
Satisfactory
Severe
Very Poor
Predicted
Good
Moderate
Poor
Satisfactory
Severe
Very Poor
3 0 0 0 0 0
0 3 0 0 0 0
0 0 3 0 0 0
0 0 0 3 0 0
0 0 0 0 0 1
0 0 0 0 0 2
3.0
2.5
2.0
1.5
1.0
0.5
0.0


Model Performance Metrics
Score
1.0
0.8
0.6
0.4
0.2
0.0
0.933
0.889
0.933
0.907
Accuracy
Precision
Recall
F1-Score


Actual vs Predicted AQI
Perfect Prediction
Predicted AQI
Actual AQI
50
100
150
200
250
300
350
400

Code :

```
import streamlit as st
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix, classification_report,
accuracy_score, precision_score, recall_score, f1_score
import numpy as np
from src.predict import predict_aqi, classify_aqi, load_model_and_scaler
from src.data_preprocessing import load_and_preprocess_data


def load_data():
    """Load the air quality dataset."""
    try:
        df = pd.read_csv('data/air_quality.csv')
        return df
    except Exception as e:
        st.error(f"Error loading data: {str(e)}")
        return None


def visualize_data(df):
    """Create data visualizations."""
    st.header("Data Visualization")

    # Basic statistics
    st.subheader("Dataset Overview")
    st.write(f"Number of samples: {len(df)}")
    st.write(f"Number of features: {len(df.columns) - 1}")  # Excluding
AQI

    col1, col2 = st.columns(2)

    with col1:
        st.subheader("Summary Statistics")
```

```
        st.dataframe(df.describe())

    with col2:
        st.subheader("Data Types")
        for col, dtype in df.dtypes.items():
            st.write(f"**{col}**: {dtype}")

    # Distribution plots
    st.subheader("Feature Distributions")
    features = ['PM2.5', 'PM10', 'NO2', 'SO2', 'CO', 'O3', 'AQI']

    fig, axes = plt.subplots(2, 4, figsize=(20, 10))
    axes = axes.ravel()

    for i, feature in enumerate(features):
        if i < len(axes):
            sns.histplot(df[feature], ax=axes[i], kde=True)
            axes[i].set_title(f'Distribution of {feature}')

    plt.tight_layout()
    st.pyplot(fig)

    # Correlation heatmap
    st.subheader("Correlation Heatmap")
    fig, ax = plt.subplots(figsize=(10, 8))
    corr_matrix = df.corr()
    sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', ax=ax)
    st.pyplot(fig)

    # Scatter plots
    st.subheader("Pollutants vs AQI")
    pollutants = ['PM2.5', 'PM10', 'NO2', 'SO2', 'CO', 'O3']

    fig, axes = plt.subplots(2, 3, figsize=(18, 12))
    axes = axes.ravel()

    for i, pollutant in enumerate(pollutants):
        sns.scatterplot(data=df, x=pollutant, y='AQI', ax=axes[i])
        axes[i].set_title(f'{pollutant} vs AQI')
```

```python
    plt.tight_layout()
    st.pyplot(fig)


def evaluate_model():
    """Evaluate the model and show metrics."""
    st.header("Model Evaluation")

    try:
        # Load data
        df = load_data()
        if df is None:
            return

        # Load model and scaler
        model, scaler = load_model_and_scaler()

        # Prepare data
        features = ['PM2.5', 'PM10', 'NO2', 'SO2', 'CO', 'O3']
        X = df[features]
        y_true_aqi = df['AQI']

        # Scale features
        X_scaled = scaler.transform(X)

        # Make predictions
        y_pred_aqi = model.predict(X_scaled)

        # Classify predictions and true values
        y_true_class = [classify_aqi(aqi) for aqi in y_true_aqi]
        y_pred_class = [classify_aqi(aqi) for aqi in y_pred_aqi]

        # Get unique classes
        classes = sorted(list(set(y_true_class + y_pred_class)))

        # Confusion Matrix
        st.subheader("Confusion Matrix")
        cm = confusion_matrix(y_true_class, y_pred_class, labels=classes)

        fig, ax = plt.subplots(figsize=(10, 8))
```

```
        sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                    xticklabels=classes, yticklabels=classes, ax=ax)
        ax.set_xlabel('Predicted')
        ax.set_ylabel('True')
        ax.set_title('Confusion Matrix')
        st.pyplot(fig)

        # Classification Report
        st.subheader("Classification Report")
        report = classification_report(y_true_class, y_pred_class,
target_names=classes, output_dict=True)

        # Convert to DataFrame for better display
        report_df = pd.DataFrame(report).transpose()
        st.dataframe(report_df.style.highlight_max(axis=0))

        # Key Metrics Visualization
        st.subheader("Model Performance Metrics")

        # Calculate metrics
        accuracy = accuracy_score(y_true_class, y_pred_class)
        precision = precision_score(y_true_class, y_pred_class,
average='weighted', zero_division=0)
        recall = recall_score(y_true_class, y_pred_class,
average='weighted', zero_division=0)
        f1 = f1_score(y_true_class, y_pred_class, average='weighted',
zero_division=0)

        # Create metrics plot
        metrics = ['Accuracy', 'Precision', 'Recall', 'F1-Score']
        values = [accuracy, precision, recall, f1]

        fig, ax = plt.subplots(figsize=(10, 6))
        bars = ax.bar(metrics, values, color=['blue', 'green', 'orange',
'red'])
        ax.set_ylim(0, 1)
        ax.set_title('Model Performance Metrics')
        ax.set_ylabel('Score')

        # Add value labels on bars
```

```
        for bar, value in zip(bars, values):
            ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() +
0.01,
                    f'{value:.3f}', ha='center', va='bottom')

        st.pyplot(fig)

        # Regression Metrics
        st.subheader("Regression Metrics")
        from sklearn.metrics import mean_absolute_error,
mean_squared_error, r2_score
        import numpy as np

        mae = mean_absolute_error(y_true_aqi, y_pred_aqi)
        rmse = np.sqrt(mean_squared_error(y_true_aqi, y_pred_aqi))
        r2 = r2_score(y_true_aqi, y_pred_aqi)

        col1, col2, col3 = st.columns(3)
        with col1:
            st.metric("MAE", f"{mae:.2f}")
        with col2:
            st.metric("RMSE", f"{rmse:.2f}")
        with col3:
            st.metric("R²", f"{r2:.3f}")

        # Predictions vs Actual
        st.subheader("Predictions vs Actual AQI")
        fig, ax = plt.subplots(figsize=(10, 6))
        ax.scatter(y_true_aqi, y_pred_aqi, alpha=0.6)
        ax.plot([y_true_aqi.min(), y_true_aqi.max()], [y_true_aqi.min(),
y_true_aqi.max()],
                'r--', lw=2, label='Perfect Prediction')
        ax.set_xlabel('Actual AQI')
        ax.set_ylabel('Predicted AQI')
        ax.set_title('Actual vs Predicted AQI')
        ax.legend()
        ax.grid(True)
        st.pyplot(fig)

    except Exception as e:
```

```
        st.error(f"Error in model evaluation: {str(e)}")
        st.warning("Please ensure the model is trained and data is
available.")


def main():
    """
    Main function to run the Streamlit application.
    """
    st.title("Air Quality Index (AQI) Prediction")
    st.markdown("""
    This application predicts the Air Quality Index (AQI) based on
pollutant concentrations.
    Enter the values for the following pollutants and click 'Predict AQI'
to get the result.
    """)

    # Create tabs
    tab1, tab2, tab3 = st.tabs(["Prediction", "Data Visualization", "Model
Evaluation"])

    with tab1:
        # Input fields for pollutants
        st.header("Pollutant Concentrations")
        col1, col2 = st.columns(2)

        with col1:
            pm25 = st.number_input("PM2.5 (µg/m³)", min_value=0.0,
value=10.0, step=0.1)
            pm10 = st.number_input("PM10 (µg/m³)", min_value=0.0,
value=20.0, step=0.1)
            no2 = st.number_input("NO2 (µg/m³)", min_value=0.0, value=5.0,
step=0.1)

        with col2:
            so2 = st.number_input("SO2 (µg/m³)", min_value=0.0, value=2.0,
step=0.1)
            co = st.number_input("CO (mg/m³)", min_value=0.0, value=0.1,
step=0.01)
            o3 = st.number_input("O3 (µg/m³)", min_value=0.0, value=15.0,
```

```
step=0.1)

        # Predict button
        if st.button("Predict AQI"):
            try:
                # Prepare input data
                pollutants = [pm25, pm10, no2, so2, co, o3]

                # Make prediction
                predicted_aqi = predict_aqi(pollutants)
                category = classify_aqi(predicted_aqi)

                # Display results
                st.success(f"Predicted AQI: {predicted_aqi:.2f}")
                st.info(f"AQI Category: {category}")

                # Additional information
                st.markdown("### AQI Categories (Indian Standards):")
                categories = {
                    "Good": "0-50",
                    "Satisfactory": "51-100",
                    "Moderate": "101-200",
                    "Poor": "201-300",
                    "Very Poor": "301-400",
                    "Severe": "401-500"
                }

                for cat, range_val in categories.items():
                    if category == cat:
                        st.markdown(f"**{cat} ({range_val})**")
                    else:
                        st.markdown(f"{cat} ({range_val})")

            except Exception as e:
                st.error(f"Error in prediction: {str(e)}")
                st.warning("Please ensure the model is trained and
available in the 'models' directory.")

    with tab2:
        df = load_data()
```

```
        if df is not None:
            visualize_data(df)

    with tab3:
        evaluate_model()


if __name__ == "__main__":
    main()
```